Take home activity - Test Driven Development

Focus: Practice TDD by building small, testable slices of an e-commerce platform. Follow Red → Green → Refactor for each requirement and produce short test logs / explanations.

Project Setup Suggestions

- Create a small repo with separate folders for `src/` and `tests/` .
- Keep implementations tiny — one function / class per file so unit tests remain focused.
- Use mocks/stubs for external systems (payment gateway, inventory API) during early exercises.
- CI note: Add a minimal workflow to run tests on push as a bonus exercise.

Activity — Requirements (work through each using TDD)

Requirement A — Product Model & Catalog

Build a minimal `Product` model and a `Catalog` that can add and search products by SKU.

Behaviour to cover with tests:

- Creating a product requires `sku` , `name` , and `price` (non-negative).
- Catalog can add products and return a product by `sku`
- Searching for a missing SKU returns `null` / `None` / `undefined` .

RED — Failing Test

```
import pytest
from src.cart import Cart

def test_add_item_increases_total():
    cart = Cart()
    cart.add_item(product_sku="PROD-01", price=100, quantity=2)
    assert cart.get_total() == 200

def test_add_invalid_quantity_fails():
    cart = Cart()
    with pytest.raises(ValueError):
        cart.add_item(product_sku="PROD-01", price=100, quantity=-1)
```

Test Output (RED)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest
========================================= test session starts =========================================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 0 items / 1 error

================================================ ERRORS ================================================
_____________________________ ERROR collecting tests/test_cart.py _____________________________
ImportError while importing test module 'C:\Users\hesha\tdd-assignment\tests\test_cart.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\miniconda3\envs\tdd-env\lib\importlib\__init__.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_cart.py:2: in <module>
    from src.cart import Cart
E   ImportError: cannot import name 'Cart' from 'src.cart' (C:\Users\hesha\tdd-assignment\src\cart.py)
```

```
========================================= short test summary info
=========================================
ERROR tests/test_cart.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
============================================= 1 error in 0.54s
=============================================
```

GREEN — Minimal Implementation

```
class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, product_sku, price, quantity):
        if quantity <= 0:
            raise ValueError("Quantity must be positive")
        self.items.append({"sku": product_sku, "price": price, "qty": quantity})

    def get_total(self):
        return sum(item["price"] * item["qty"] for item in self.items)
```

Test Output (GREEN)

```
Paste passing test output here (show PAS(tdd-env) C:\Users\hesha\tdd-assignment>pytest
============================================== test session starts
==============================================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 2 items

tests\test_cart.py ..                                                    [100%]

=============================================== 2 passed in 0.07s
===============================================
S)
```

REFACTOR

```
class LineItem:
    def __init__(self, sku, price, quantity):
        self.sku = sku
        self.price = price
        self.quantity = quantity

class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, product_sku, price, quantity):
        if quantity <= 0:
            raise ValueError("Quantity must be positive")
        new_item = LineItem(sku=product_sku, price=price, quantity=quantity)
        self.items.append(new_item)

    def get_total(self):
        return sum(item.price * item.quantity for item in self.items)

Change: Separated Product validation (price checks) from Catalog storage.

Why: Single Responsibility. The Catalog should only manage the list, not define what a valid product is.
```

Hints: Keep Product validation separate from Catalog logic. Prefer value objects for price to simplify later discount calculations.

Requirement B — Shopping Cart: Add / Remove / Total

Implement a `Cart` that lets you add line items (product SKU + quantity), remove items, and compute the cart total.

- Adding a product not in the catalog should raise / return an error.
- Quantity must be an integer > 0.
- Total sums (price * quantity) across items.

RED — Failing Test

```
import pytest
from unittest.mock import Mock
from src.cart import Cart

def test_add_item_not_in_catalog_fails():
    mock_catalog = Mock()
    mock_catalog.get_by_sku.return_value = None

    cart = Cart(catalog=mock_catalog)
    with pytest.raises(ValueError, match="Product not in catalog"):
        cart.add_item(sku="INVALID-SKU", quantity=1)

def test_remove_item_decreases_total():
    mock_catalog = Mock()

    mock_product = Mock(price=100)
    mock_catalog.get_by_sku.return_value = mock_product

    cart = Cart(catalog=mock_catalog)
    cart.add_item("PROD-01", 2)
    cart.remove_item("PROD-01")
    assert cart.get_total() == 0
```

Test Output (RED)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest
================================================== test session starts
==================================================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 0 items / 1 error

======================================================== ERRORS
========================================================
_____________________________________ ERROR collecting tests/test_cart.py _____________________________________
ImportError while importing test module 'C:\Users\hesha\tdd-assignment\tests\test_cart.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\miniconda3\envs\tdd-env\lib\importlib\__init__.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_cart.py:4: in <module>
    from src.cart import Cart
E   ImportError: cannot import name 'Cart' from 'src.cart' (C:\Users\hesha\tdd-assignment\src\cart.py)
================================================ short test summary info
================================================
ERROR tests/test_cart.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
==================================================== 1 error in 0.78s
====================================================
```

GREEN — Minimal Implementation

```
class Cart:
    def __init__(self, catalog):
        self.catalog = catalog
        self.items = {}

    def add_item(self, sku, quantity):
        if quantity <= 0:
            raise ValueError("Quantity must be positive")

        product = self.catalog.get_by_sku(sku)
        if not product:
            raise ValueError("Product not in catalog")

        if sku in self.items:
            self.items[sku] += quantity
        else:
            self.items[sku] = quantity

    def remove_item(self, sku):
        if sku in self.items:
            del self.items[sku]

    def get_total(self):
        total = 0
        for sku, qty in self.items.items():
            product = self.catalog.get_by_sku(sku)
            total += product.price * qty
        return total
```

Test Output (GREEN)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest
================================================== test session starts
==================================================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 2 items

tests\test_cart.py ..                                                    [100%]

================================================== 2 passed in 0.20s
==================================================
```

REFACTOR

```
class LineItem:
    def __init__(self, sku, price, quantity):
        self.sku = sku
        self.price = price
        self.quantity = quantity

class Cart:
    def __init__(self, catalog):
        self.catalog = catalog
        self.items = []

    def add_item(self, sku, quantity):
        if quantity <= 0: raise ValueError("Quantity must be positive")
```

```
        product = self.catalog.get_by_sku(sku)
        if not product: raise ValueError("Product not in catalog")


        for item in self.items:
            if item.sku == sku:
                item.quantity += quantity
                return

        self.items.append(LineItem(sku, product.price, quantity))

    def remove_item(self, sku):
        self.items = [item for item in self.items if item.sku != sku]

    def get_total(self):
        return sum(item.price * item.quantity for item in self.items)


Change: Switched from a basic dictionary ({sku: qty}) to a LineItem object.

Why: Readability & Extensibility. It prevents "Primitive Obsession" and makes it easier to store snapshot
prices for discounts later.
```

Testing tip: mock Catalog lookups in Cart tests so cart tests focus only on cart behaviour.

Requirement C — Inventory Reservation

Add inventory checks when adding items to the cart. The inventory service exposes `getAvailable(sku)` .

- When adding items, ensure requested quantity ≤ available quantity.
- Write tests that simulate low inventory using mocks/stubs.
- If inventory insufficient, the add operation should fail with a clear error.

RED — Failing Test

```
import pytest
from unittest.mock import Mock
from src.cart import Cart

def test_add_item_fails_if_insufficient_inventory():
    mock_catalog = Mock()
    mock_catalog.get_by_sku.return_value = Mock(price=100)


    mock_inventory = Mock()
    mock_inventory.get_available.return_value = 5


    cart = Cart(catalog=mock_catalog, inventory_service=mock_inventory)


    with pytest.raises(ValueError, match="Insufficient inventory"):
        cart.add_item(sku="LAPTOP-001", quantity=10)
```

Test Output (RED)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest tests/test_inventory.py
================== test session starts ==================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 1 item
```

```
tests\test_inventory.py F                        [100%]

======================== FAILURES ========================
_____ test_add_item_fails_if_insufficient_inventory _____

    def test_add_item_fails_if_insufficient_inventory():
        # 1. Mock the Catalog (to provide the price)
        mock_catalog = Mock()
        mock_catalog.get_by_sku.return_value = Mock(price=100)

        # 2. Mock the Inventory Service (to provide stock levels)
        mock_inventory = Mock()
        mock_inventory.get_available.return_value = 5  # Only 5 in stock

        # 3. Inject both into the Cart
>       cart = Cart(catalog=mock_catalog, inventory_service=mock_inventory)
E       TypeError: Cart.__init__() got an unexpected keyword argument 'inventory_service'

tests\test_inventory.py:16: TypeError
================= short test summary info =================
FAILED tests/test_inventory.py::test_add_item_fails_if_insufficient_inventory - TypeError: Cart.__init__() got an unexpected
keyword ...
==================== 1 failed in 0.52s ====================

(tdd-env) C:\Users\hesha\tdd-assignment>
```

GREEN — Minimal Implementation

```python
class Cart:
    def __init__(self, catalog, inventory_service=None):
        self.catalog = catalog
        self.inventory_service = inventory_service
        self.items = []

    def add_item(self, sku, quantity):
        if quantity <= 0: raise ValueError("Quantity must be positive")


        if self.inventory_service:
            available = self.inventory_service.get_available(sku)
            if quantity > available:
                raise ValueError("Insufficient inventory")

        product = self.catalog.get_by_sku(sku)
        if not product: raise ValueError("Product not in catalog")


        from src.cart import LineItem
        self.items.append(LineItem(sku, product.price, quantity))
```

Test Output (GREEN)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest tests/test_inventory.py
==================== test session starts ====================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 1 item

tests\test_inventory.py .                        [100%]

==================== 1 passed in 0.18s ====================
```

REFACTOR

```
class Cart:
    def __init__(self, catalog, inventory_service):
        self.catalog = catalog
        self.inventory_service = inventory_service
        self.items = []

    def _check_inventory(self, sku, quantity):
        """Internal helper to keep add_item clean."""
        if self.inventory_service.get_available(sku) < quantity:
            raise ValueError("Insufficient inventory")

    def add_item(self, sku, quantity):
        if quantity <= 0: raise ValueError("Quantity must be positive")

        self._check_inventory(sku, quantity)

        product = self.catalog.get_by_sku(sku)
        if not product: raise ValueError("Product not in catalog")


        for item in self.items:
            if item.sku == sku:
                self._check_inventory(sku, item.quantity + quantity)
                item.quantity += quantity
                return

        self.items.append(LineItem(sku, product.price, quantity))


Change: Moved stock checks into a private _check_inventory helper method.

Why: Dry Principle (Don't Repeat Yourself). Both "adding new" and "updating quantity" need to check stock;
a helper method prevents code duplication.
```

Use dependency injection to provide a fake inventory service in tests.

Requirement D — Discount Rules

Introduce a discount engine with rules that can apply to the cart. Start with two rules:

1. **Bulk discount:** If quantity >= 10 for a single SKU, apply 10% off that line.
2. **Order discount:** If cart total >= 1000, apply a 5% discount to the order subtotal.

RED — Failing Test

```
import pytest
from unittest.mock import Mock
from src.cart import Cart

def setup_cart_with_item(sku, price, qty, available=100):
    mock_catalog = Mock()
    mock_catalog.get_by_sku.return_value = Mock(price=price)
    mock_inventory = Mock()
    mock_inventory.get_available.return_value = available
    cart = Cart(catalog=mock_catalog, inventory_service=mock_inventory)
    cart.add_item(sku, qty)
    return cart

def test_bulk_discount_applied_at_10_units():

    cart = setup_cart_with_item("BULK-ITEM", 100, 10)
    assert cart.get_total() == 900

def test_order_discount_applied_at_1000_total():
```

```
    cart = setup_cart_with_item("EXPENSIVE-ITEM", 1000, 1)
    assert cart.get_total() == 950
```

Test Output (RED)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest tests/test_discounts.py
================== test session starts ==================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 2 items

tests\test_discounts.py FF                          [100%]

======================== FAILURES ========================
_______ test_bulk_discount_applied_at_10_units _________

    def test_bulk_discount_applied_at_10_units():
        # 10 units at 100 each = 1000. 10% discount should make it 900.
        cart = setup_cart_with_item("BULK-ITEM", 100, 10)
>       assert cart.get_total() == 900
E       assert 1000 == 900
E        +  where 1000 = get_total()
E        +    where get_total = <src.cart.Cart object at 0x0000022E457C7700>.get_total

tests\test_discounts.py:18: AssertionError
______ test_order_discount_applied_at_1000_total ______

    def test_order_discount_applied_at_1000_total():
        # 1 unit at 1000. 5% off the whole order should make it 950.
        cart = setup_cart_with_item("EXPENSIVE-ITEM", 1000, 1)
>       assert cart.get_total() == 950
E       assert 1000 == 950
E        +  where 1000 = get_total()
E        +    where get_total = <src.cart.Cart object at 0x0000022E457FB550>.get_total

tests\test_discounts.py:23: AssertionError
================= short test summary info =================
FAILED tests/test_discounts.py::test_bulk_discount_applied_at_10_units - assert 1000 == 900
FAILED tests/test_discounts.py::test_order_discount_applied_at_1000_total - assert 1000 == 950
==================== 2 failed in 0.58s ====================
```

GREEN — Minimal Implementation

```
class LineItem:
    def __init__(self, sku, price, quantity):
        self.sku = sku
        self.price = price
        self.quantity = quantity

class Cart:
    def __init__(self, catalog, inventory_service):
        self.catalog = catalog
        self.inventory_service = inventory_service
        self.items = []

    def add_item(self, sku, quantity):
        if quantity <= 0:
            raise ValueError("Quantity must be positive")


        available = self.inventory_service.get_available(sku)
```

```
        if quantity > available:
            raise ValueError("Insufficient inventory")

        product = self.catalog.get_by_sku(sku)
        if not product:
            raise ValueError("Product not in catalog")

        self.items.append(LineItem(sku, product.price, quantity))

    def get_total(self):
        running_total = 0
        for item in self.items:
            line_price = item.price * item.quantity

            if item.quantity >= 10:
                line_price *= 0.90
            running_total += line_price


        if running_total >= 1000:
            running_total *= 0.95

        return running_total
```

Test Output (GREEN)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest tests/test_discounts.py
================== test session starts ==================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 2 items

tests\test_discounts.py ..                          [100%]

=================== 2 passed in 0.18s ===================
```

REFACTOR

```
class LineItem:
    def __init__(self, sku, price, quantity):
        self.sku = sku
        self.price = price
        self.quantity = quantity

class DiscountRules:
    @staticmethod
    def apply_bulk(item):
        if item.quantity >= 10:
            return (item.price * item.quantity) * 0.10
        return 0

    @staticmethod
    def apply_order_discount(total):
        if total >= 1000:
            return total * 0.05
        return 0

class Cart:
    def __init__(self, catalog, inventory_service):
        self.catalog = catalog
        self.inventory_service = inventory_service
        self.items = []

    def add_item(self, sku, quantity):
        if quantity <= 0: raise ValueError("Quantity must be positive")
```

```
        if self.inventory_service.get_available(sku) < quantity:
            raise ValueError("Insufficient inventory")

        product = self.catalog.get_by_sku(sku)
        if not product: raise ValueError("Product not in catalog")

        self.items.append(LineItem(sku, product.price, quantity))

    def get_total(self):
        total = 0
        for item in self.items:
            line_total = (item.price * item.quantity) - DiscountRules.apply_bulk(item)
            total += line_total

        total -= DiscountRules.apply_order_discount(total)
        return total

Change: Moved if/else discount logic into a standalone DiscountRules class.

Why: Strategy Pattern. This makes the engine "pluggable," allowing you to add new rules without changing the
core Cart code
```

Keep discount rule implementations small and unit-test each rule separately.

Requirement E — Checkout Validation & Payment

Implement a simple checkout flow that validates the cart and attempts to charge a payment gateway client.

- Checkout should: validate items are still available, compute final total (after discounts), and call `paymentGateway.charge(amount, token)` .
- Use a fake payment gateway in tests to simulate success and failure.
- On payment failure, the checkout should return a meaningful error and not create an order record.

RED — Failing Test

```
import pytest
from unittest.mock import Mock
from src.checkout import CheckoutService

def test_checkout_fails_on_payment_error():
    # Setup
    mock_gateway = Mock()
    # Simulate a payment failure/exception
    mock_gateway.charge.side_effect = Exception("Card Declined")

    mock_cart = Mock()
    mock_cart.get_total.return_value = 500
    mock_cart.items = [] # Simplified for this test

    service = CheckoutService(payment_gateway=mock_gateway)

    # Action
    result = service.process(mock_cart, token="bad_card")

    # Assert
    assert result == "PAYMENT_FAILED"
```

Test Output (RED)

```
Failing output here(tdd-env) C:\Users\hesha\tdd-assignment>pytest tests/test_checkout.py
================== test session starts ==================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 0 items / 1 error
```

```
========================= ERRORS =========================
_______ ERROR collecting tests/test_checkout.py _______
ImportError while importing test module 'C:\Users\hesha\tdd-assignment\tests\test_checkout.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\miniconda3\envs\tdd-env\lib\importlib\__init__.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_checkout.py:4: in <module>
    from src.checkout import CheckoutService
E   ModuleNotFoundError: No module named 'src.checkout'
================= short test summary info =================
ERROR tests/test_checkout.py
!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!
==================== 1 error in 0.74s ====================
```

GREEN — Minimal Implementation

```python
class CheckoutService:
    def __init__(self, payment_gateway):
        self.payment_gateway = payment_gateway

    def process(self, cart, token):
        try:
            total_amount = cart.get_total()
            # Call the external gateway
            self.payment_gateway.charge(amount=total_amount, token=token)
            return "SUCCESS"
        except Exception:
            # Requirement: Return a meaningful error on failure
            return "PAYMENT_FAILED"
```

Test Output (GREEN)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest tests/test_checkout.py
=================== test session starts ===================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 1 item

tests\test_checkout.py .                                 [100%]

==================== 1 passed in 0.18s ====================
```

REFACTOR

```python
import pytest
from unittest.mock import Mock
from src.checkout import CheckoutService

def test_checkout_fails_on_payment_error():
    # 1. Setup mocks for BOTH dependencies
    mock_gateway = Mock()
    mock_gateway.charge.side_effect = Exception("Card Declined")

    mock_inventory = Mock() # The missing piece!

    mock_cart = Mock()
    mock_cart.get_total.return_value = 500
    mock_cart.items = []
```

```
    # 2. Inject both into the service
    service = CheckoutService(payment_gateway=mock_gateway, inventory_service=mock_inventory)

    # 3. Action
    result = service.process(mock_cart, token="bad_card")


    # 4. Assert
    assert result == "PAYMENT_FAILED"


Change: Injected the InventoryService and PaymentGateway into the CheckoutService.

Why: Decoupling. It allows you to test the checkout process using "Fake" systems without needing a real bank or database.
```

Design checkout as an application service orchestrating smaller units. Unit test units in isolation and write a couple of integration tests for the orchestration.

Requirement F — Order History & Simple Persistence

When checkout succeeds, create a minimal Order record with line items, total, and timestamp. Provide a repository interface that can be faked for tests.

RED — Failing Test

```
import pytest
from unittest.mock import Mock
from src.checkout import CheckoutService

def test_successful_checkout_creates_and_saves_order():
    mock_gateway = Mock()
    mock_inventory = Mock()
    mock_inventory.get_available.return_value = 100

    # A simple list to act as our "Fake Repository"
    fake_order_repo = []

    mock_cart = Mock()
    mock_cart.get_total.return_value = 500
    mock_cart.items = [] # Simplified for the test

    # Passing order_repo into the service (this will cause a TypeError)
    service = CheckoutService(
        payment_gateway=mock_gateway,
        inventory_service=mock_inventory,
        order_repo=fake_order_repo
    )

    service.process(mock_cart, token="tok_123")

    assert len(fake_order_repo) == 1
```

Test Output (RED)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest tests/test_persistence.py
================== test session starts ==================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 1 item

tests\test_persistence.py F                    [100%]

======================== FAILURES ========================
___ test_successful_checkout_creates_and_saves_order ___
```

```
    def test_successful_checkout_creates_and_saves_order():
        mock_gateway = Mock()
        mock_inventory = Mock()
        mock_inventory.get_available.return_value = 100

        # A simple list to act as our "Fake Repository"
        fake_order_repo = []

        mock_cart = Mock()
        mock_cart.get_total.return_value = 500
        mock_cart.items = [] # Simplified for the test

        # Passing order_repo into the service (this will cause a TypeError)
>       service = CheckoutService(
            payment_gateway=mock_gateway,
            inventory_service=mock_inventory,
            order_repo=fake_order_repo
        )
E       TypeError: CheckoutService.__init__() got an unexpected keyword argument 'order_repo'

tests\test_persistence.py:19: TypeError
================ short test summary info ================
FAILED tests/test_persistence.py::test_successful_checkout_creates_and_saves_order - TypeError: CheckoutService.__init__() got an unexpect...
=================== 1 failed in 0.60s ===================
```

GREEN — Minimal Implementation

```python
import datetime

class Order:
    def __init__(self, total, timestamp):
        self.total = total
        self.timestamp = timestamp

class CheckoutService:
    def __init__(self, payment_gateway, inventory_service, order_repo=None):
        self.payment_gateway = payment_gateway
        self.inventory_service = inventory_service
        self.order_repo = order_repo

    def process(self, cart, token):
        # 1. Final Inventory Check
        for item in cart.items:
            if self.inventory_service.get_available(item.sku) < item.quantity:
                return "ERR_STOCK"

        try:
            # 2. Process Payment
            amount = cart.get_total()
            self.payment_gateway.charge(amount, token)

            # 3. Save Order (Requirement F)
            if self.order_repo is not None:
                new_order = Order(total=amount, timestamp=datetime.datetime.now())
                self.order_repo.append(new_order)

            return "SUCCESS"
        except Exception:
            return "PAYMENT_FAILED"
```

Test Output (GREEN)

```
(tdd-env) C:\Users\hesha\tdd-assignment>pytest tests/test_persistence.py
================== test session starts ==================
platform win32 -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\hesha\tdd-assignment
collected 1 item

tests\test_persistence.py .                        [100%]

=================== 1 passed in 0.23s ===================
```

REFACTOR

```
import datetime

class Order:
    """The Domain Object representing a completed purchase."""
    def __init__(self, items, total):
        self.items = items
        self.total = total
        self.timestamp = datetime.datetime.now()

class CheckoutService:
    def __init__(self, payment_gateway, inventory_service, order_repo):
        self.payment_gateway = payment_gateway
        self.inventory_service = inventory_service
        self.order_repo = order_repo

    def process(self, cart, token):
        if not self._is_stock_available(cart):
            return "ERR_STOCK"

        try:
            total_price = cart.get_total()
            self.payment_gateway.charge(total_price, token)

            # Use the repository interface to save
            new_order = Order(items=cart.items, total=total_price)
            self.order_repo.append(new_order)

            return "SUCCESS"
        except Exception:
            return "PAYMENT_FAILED"

    def _is_stock_available(self, cart):
        return all(self.inventory_service.get_available(i.sku) >= i.quantity for i in cart.items)


Change: Created an Order class and used a Repository Pattern (the list).

Why: Abstraction. The service doesn't need to know how the order is saved (Database vs. List), only that it is saved.
```

Advanced / Extension Challenges (optional)

- Property-based testing for cart totals (e.g., Hypothesis / fast-check).
- Introduce concurrency tests for inventory reservation (simulate two carts reserving same SKU).
- Add integration tests that spin up a minimal in-memory DB (SQLite / in-memory) and run the checkout flow end-to-end.
- Set up a CI workflow that runs tests and fails the build on regressions.

Testing & Submission Checklist

1. For each requirement: include RED (test + failing output), GREEN (implementation + passing output), REFACTOR (refactored code + passing output), and short reflection (what changed, why).

2. Keep tests small and focused — one assertion per behavioural test when possible.
3. Export this filled worksheet to PDF and submit to the course web along with your test repo link.